

Assignment 2

Aaradhya (230011) · Anjali Patra (230148) · Marisha (230637) · Prakriti (230770) · Pranaya (230324) Group 72

1. Code Logic

1.1. Domain Decomposition and Process Mapping

The global 3D domain of size $(p_x \times n_x) \times (p_y \times n_y) \times (p_z \times n_z)$ is partitioned across $P = p_x \times p_y \times p_z$ MPI processes, each owning $n_x \times n_y \times n_z$ interior points per field. A process of rank r maps to 3D process coordinates:

$$r_x = r \bmod p_x, \quad r_y = \lfloor r/p_x \rfloor \bmod p_y, \quad r_z = \lfloor r/(p_x p_y) \rfloor$$

Six face neighbours $(\pm x, \pm y, \pm z)$ are derived from these coordinates. Processes on the global boundary use `MPI_PROC_NULL`, which silently no-ops all associated sends and receives.

1.2. Extended Array and Memory Layout

Each process allocates an extended buffer of size $(n_x+2h) \times (n_y+2h) \times (n_z+2h)$ per field for both `cur` and `nxt` arrays, where $h = (d-1)/6$ ($h = 1$ for $d = 7$). The h -wide ghost shell stores incoming halo data. A macro `IDX(x, y, z) = x + (n_x+2h)(y + (n_y+2h)z)` linearises 3D indices into the padded 1D array, keeping x -direction accesses stride-1 in memory for cache efficiency. Interior data is placed at offsets `[1x+h] [1y+h] [1z+h]` following the prescribed `srand(seed)` loop.

1.3. MPI Derived Datatypes

Three `MPI_Type_indexed` datatypes are constructed *once* before the time loop, each describing a face as non-contiguous blocks in the live array:

- **X-face:** $n_y \times n_z$ blocks of h doubles (row stride n_x+2h) - YZ-slab.
- **Y-face:** $h \times n_z$ blocks of n_x doubles (same row stride) - XZ-slab.
- **Z-face:** $n_y \times h$ blocks of n_x doubles (layer stride $(n_x+2h)(n_y+2h)$) - XY-slab.

Since displacements address the live array directly, each face exchange uses a single `MPI_Isend/MPI_Irecv` with *zero* temporary copies. This avoids explicit strided memory packing, which would otherwise degrade cache locality, increase memory bandwidth pressure, and allows the MPI library to transfer data more efficiently directly from memory.

1.4. Halo Exchange

At the start of each step, all $6F$ `MPI_Irecv` calls are posted first, landing ghost data into the outer shells of `cur[fi]` which is followed by $6F$ `MPI_Isend` calls from the inner boundary layers. Directional tags (e.g. `TAG_XM` on a send toward $-x$, matched by the recipient) prevent message mismatches. All $12F$ request handles are gathered for a single `MPI_Waitall`.

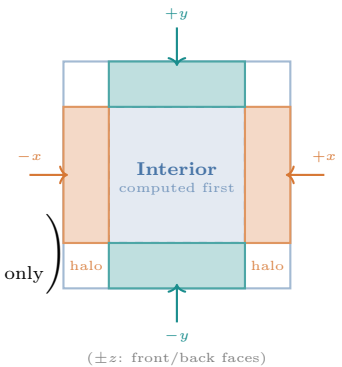
1.5. Overlapped Stencil Computation

After posting all non-blocking communication, the *interior* sub-domain ($lx \in [h, n_x-h]$, $ly \in [h, n_y-h]$, $lz \in [h, n_z-h]$) is updated using only locally held data, while halo exchanges run in the background. `MPI_Waitall` is called only after the interior update finishes; then the boundary ring is updated using the arrived halo data.

For each point P at global coordinates (g_x, g_y, g_z) , the d -point stencil is:

$$\text{nxt}(P) = \frac{1}{m} \left(\text{cur}(P) + \sum_{s=1}^h \left[\text{cur}(P \pm s \hat{x}) + \text{cur}(P \pm s \hat{y}) + \text{cur}(P \pm s \hat{z}) \right]_{\text{valid only}} \right)$$

where m is the number of terms included (self + valid neighbours). “Valid” means the neighbour lies within the global domain. Boundary processes with fewer stencil arms divide by the actual $m \leq 1+6h$, requiring no extra communication, only integer comparisons of global coordinates against domain extents.



2D cross-section. Orange: x -halos. Teal: y -halos. Blue: interior, computed while halos are in-flight.

1.6. Pointer Swap, Isovalue Counting, and Output

After each step, `cur` and `nxt` are swapped in $O(1)$ (two pointer assignments, no data copy). Each process counts axis-aligned isovalue *crossings* per field: $(v_0 - \text{iso})(v_1 - \text{iso}) < 0$ for each $+x, +y, +z$ neighbour pair within the global domain. Local counts are aggregated to rank 0 via one `MPI_Reduce` (`MPI_LONG`, `MPI_SUM`) and printed. After all T steps, `MPI_Barrier` synchronises all ranks before rank 0 prints total wall-clock time via `MPI_Wtime`.

2. Code Optimizations

Table 1. Summary of optimizations, alternatives, and quantified benefits

Opt.	Without	With	Primary Benefit
O1: Derived datatypes	Manual pack/unpack buffers	Direct send from live array	−5.5 MB traffic/field/step
O2: Non-blocking comms	Blocking <code>Sendrecv</code>	<code>Isend/Irecv</code>	6 faces injected simultaneously
O3: Compute overlap	Wait then compute all	Compute interior while waiting	≥95% work overlapped
O4: Pointer swap	Full array copy	Two pointer assignments	−221 MB copy/step ($n_x = 240$)
O5: Batched reduce	F separate reductions	One <code>MPI_Reduce</code>	Fewer collectives per step
O6: Dynamic denominator	Fixed divide by d	Divide by actual m	Correct boundary averages

O1. Derived datatypes (zero-copy face communication): Without derived datatypes, sending a face requires a manual `memcpy` of non-contiguous rows into a flat buffer (pack), followed by a matching unpack, two strided traversals that pollute cache and consume memory bandwidth. With `MPI_Type_indexed`, the MPI library internally handles the scatter/gather of non-contiguous data, allowing direct communication from the array without additional memory copies. For $n_x = 240$, $h = 1$, each face is $240^2 = 57,600$ doubles (≈ 460 KB); eliminating pack/unpack for 6 faces saves approx 5.5 MB of memory traffic per field per step.

O2. Non-blocking communication: All six face exchanges per field are posted simultaneously before any computation. Posting `MPI_Irecv` first avoids potential deadlock and lets the MPI runtime inject all messages into the network concurrently, exploiting bidirectional bandwidth across all six directions at once.

O3. Compute-communication overlap: The interior-first update runs while halos are in transit. Overlap efficiency:

$$f_{\text{interior}} = \frac{(n_x - 2h)^3}{n_x^3} \approx 95.1\% \ (n_x = 120), \quad 97.5\% \ (n_x = 240)$$

Without overlap, each step incurs an idle gap equal to the inter-node round-trip latency (~ 5 ms on PARAM Rudra’s InfiniBand), adding ~ 25 ms across $T = 5$ steps which is significant relative to the ~ 0.9 s total for $n_x = 120$.

O4. $O(1)$ pointer swap: Swapping `cur/nxt` with two pointer assignments avoids copying $0F \times n_x n_y n_z \times 8$ bytes: 27.6 MB for $n_x = 120$ and 221 MB for $n_x = 240$ per step.

O5. & O6. are described in Table 1.

3. Results

All experiments ran on PARAM Rudra ($d = 7$, $T = 5$, $F = 2$, `seed`=1000, `isovalue`=500), each configuration executed 5 independent times via separate SLURM jobs on the `standard` partition. Timing uses `MPI_Barrier`+`MPI_Wtime` enclosing all T steps.

Table 2. Execution times (s) and weak-scaling efficiency $E = T_{P_{\min}}/T_P$.

n_x	P	Run 1	Run 2	Run 3	Run 4	Run 5	Mean $\pm$ Std (s)	Efficiency E
120	32	0.931	0.838	0.838	0.834	0.832	0.854 ± 0.038	1.000 (ref)
	48	0.924	0.929	0.927	0.926	0.928	0.927 ± 0.002	0.921
	64	0.928	0.918	0.928	0.918	0.929	0.924 ± 0.005	0.924
	96	0.930	0.930	0.924	0.938	0.943	0.933 ± 0.007	0.915
240	32	8.200	7.014	6.942	6.977	6.987	7.224 ± 0.490	1.000 (ref)
	48	7.945	7.975	8.024	7.985	7.972	7.980 ± 0.029	0.905
	64	7.976	7.851	7.918	7.873	7.982	7.920 ± 0.053	0.912
	96	7.993	7.985	7.871	8.000	7.941	7.958 ± 0.048	0.907

Table 3. Per-configuration results: isovalue crossings (F1, F2) at each time step.

P	Size	T=1 (F1, F2)	T=2 (F1, F2)	T=3 (F1, F2)	T=4 (F1, F2)	T=5 (F1, F2)
32	120	1233268, 1234820	83292, 84753	14780, 15103	2882, 2762	462, 506
32	240	36096696, 36090335	15565227, 15559788	9419690, 9418644	6358527, 6355411	4803067, 4795211
48	120	1751498, 1753126	85355, 86537	14731, 15053	2840, 2728	432, 476
48	240	38181467, 38175802	15634506, 15631669	9407502, 9406264	6406003, 6402995	4852240, 4847813
64	120	2385268, 2386820	86895, 88121	14780, 15103	2882, 2762	462, 506
64	240	40701836, 40695722	15658445, 15655913	9419690, 9418644	6358527, 6355411	4803067, 4795211
96	120	3537098, 3538726	88852, 90127	14731, 15053	2840, 2728	432, 476
96	240	45323538, 45317937	15734368, 15734153	9407502, 9406264	6406003, 6402995	4852240, 4847813

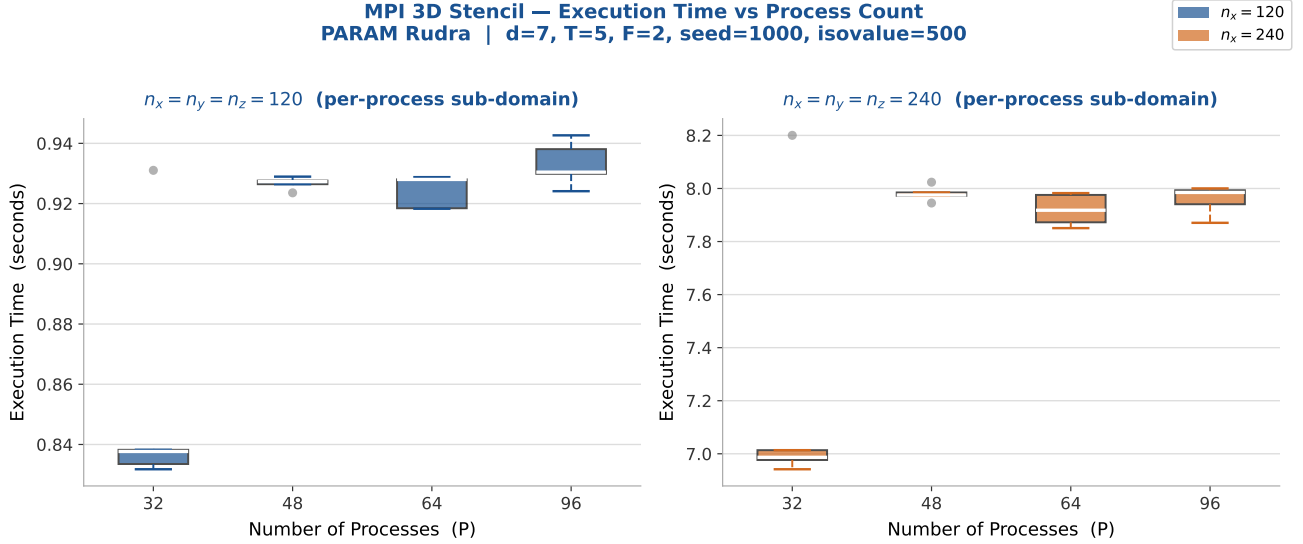


Figure 1. Execution time boxplots (5 runs each) for all 8 configurations on PARAM Rudra. Left: $n_x = n_y = n_z = 120$. Right: $n_x = n_y = n_z = 240$. Box: median and IQR; whiskers: $1.5 \times \text{IQR}$; individual points mark outliers.

4. Observations

4.1. Weak Scaling and Efficiency

Since n_x, n_y, n_z are per-process sizes fixed across all P , total problem size grows with P , this is a *weak scaling* experiment. Weak-scaling efficiency $E = T_{P_{\min}}/T_P$ ($P_{\min} = 32$) is shown in Table 2. For $n_x = 120$, E falls from 1.000 to 0.915 at $P = 96$ (8.5% overhead), attributable to a deeper `MPI_Reduce` tree and more inter-node face exchanges. For $n_x = 240$, E remains ≈ 0.91 across $P = 48$ – 96 with $< 1\%$ variation, confirming the implementation is compute-bound at larger sub-domains and scales well in weak-scaling sense.

4.2. Compute-Bound vs. Communication-Bound Regimes

Moving from $n_x = 120$ to $n_x = 240$ increases per-process computation by $8 \times (240^3/120^3)$ but face area by only $4 \times$, halving the communication-to-computation ratio. Observed runtime increases by 8.5 times (e.g. $P = 48$: $0.927 \rightarrow 7.980$ s), near the ideal $8 \times$, confirming compute dominance at $n_x = 240$. At $n_x = 120$, lighter computation exposes fixed collective costs, producing the mild upward trend with P and slightly wider boxplot spreads.

4.3. Variance Reduction at Larger Sub-domains

The $n_x = 240$ boxplots are visibly tighter ($\text{std} \leq 0.054$ s, $< 0.7\%$ of mean) compared to $n_x = 120$ (std up to 0.038 s, 4% of mean). Larger workloads amortise fixed system noise - OS scheduler jitter, MPI variability, network contention, which represent a constant absolute overhead. When computation dominates, this noise becomes a negligible fraction of total time, yielding more reproducible measurements. At $n_x = 240$, the results vary less because the computation is larger and system noise has less effect.

4.4. Node-Level Variability at $P = 32$, $n_x = 240$

Run 1 (8.200 s on `rdcn25`) is 17% slower than runs 2–5 (6.97 s on `rdcn30--33`). Since all 32 processes fit on one node, no inter-node traffic is involved; the outlier reflects intra-node factors (co-located job, cold memory state).

This underscores the importance of multiple trials and boxplot visualisation over single-point timing.

4.5. Isovalue Crossing Counts: Rapid Smoothing

Crossing counts decrease sharply over time, with a large drop in the initial step followed by progressively smaller reductions in later steps (Table 3). This is because the stencil acts as an averaging operator: each iteration smooths the scalar field and reduces sharp local variations. As a result, fewer neighboring grid points lie on opposite sides of the isovalue, leading to fewer crossings. The rapid initial drop indicates that most high-frequency variation is removed in the first step, while later steps progressively refine the smoothing.

Crossing counts also differ across configurations (e.g. 1,233,268 at $P = 32$ vs. 3,537,098 at $P = 96$ for $n_x = 120$, step 1), which is expected. Since the initialisation scales values by `myrank+1`, larger process counts generate a more varied global dataset, resulting in more crossings. The identical counts across all 5 runs within each configuration confirm that the implementation is deterministic and correct.

4.6. Limitations and Scalability Considerations

- **Small sub-domains at high P :** As $n_x \rightarrow 2h$, the interior fraction $f_{\text{interior}} \rightarrow 0$, eliminating compute-communication overlap (O3) and making the implementation latency-bound.
- **Derived datatype overhead at small faces:** The `MPI_Type_indexed` scatter/gather carries a fixed setup cost per call; at small face sizes this may exceed a simple `memcpy` pack.
- **Collective bottleneck at large P :** `MPI_Reduce` at very large P incurs a logarithmic tree cost that may become non-negligible relative to local computation.
- **Load balance:** The decomposition assumes p_x, p_y, p_z exactly divide the domain. Non-divisible sizes yield unequal sub-domains, causing load imbalance and idle time at `MPI_Waitall`.